

Detecção Automática de Pragas em Plantações de Café Utilizando Redes Neurais Convolucionais

**Eduardo Honório Friaca (10408959), Felipe Jiao (10408852), Hao Yue Zheng
(10408948), Samuel Zheng (10395781)**

**Universidade Presbiteriana Mackenzie
São Paulo - SP – Brazil**

²FCI - Faculdade de Computação e Informática

Curso de Ciência da Computação

Github: <https://github.com/mauriciohao/IA> ;

Colab: <https://drive.google.com/file/d/1HmCGqqBy1zo6-bvg-EGS6ST3T6lf3cns/view?usp=sharing> ;

Youtube: <https://youtu.be/qoXE-rwSb4k?si=xShc9Go7vrDK2IE0>

RESUMO

O controle de pragas agrícolas é um dos principais desafios da agricultura moderna, impactando diretamente a produtividade e a qualidade dos alimentos. Neste trabalho, é desenvolvido um sistema de classificação automática de pragas em imagens de plantas utilizando técnicas de Inteligência Artificial, especificamente redes neurais convolucionais (CNNs) com aprendizado por transferência (transfer learning). O estudo utiliza o conjunto de dados IP102, que contém imagens de diferentes espécies de insetos-praga, organizadas em múltiplas classes. As imagens são organizadas, pré-processadas e divididas em subconjuntos de treinamento, validação e teste. Em seguida, é adaptado um modelo pré-treinado ResNet-18, ajustando-se a camada final para o número de classes do problema. O modelo é treinado e avaliado em ambiente Google Colab, utilizando a biblioteca PyTorch. São analisadas métricas de desempenho, como acurácia no conjunto de teste, bem como a matriz de confusão, permitindo avaliar o padrão de acertos e erros entre as classes. Os resultados obtidos indicam que a abordagem de deep learning é capaz de aprender padrões visuais relevantes para a identificação de pragas, apresentando desempenho promissor como ferramenta de apoio à tomada de decisão em contextos agrícolas. Por fim, são discutidas limitações do modelo e possíveis melhorias futuras.

Palavras-chave: Inteligência Artificial; Visão Computacional; Deep Learning; Pragas Agrícolas; Redes Neurais Convolucionais.

ABSTRACT

Pest control is one of the main challenges of modern agriculture, directly affecting productivity and food quality. In this work, we develop an automatic pest classification system for plant images using Artificial Intelligence techniques, specifically Convolutional Neural Networks (CNNs) with transfer learning. We use the IP102 dataset, which contains images of multiple insect pest species organized into several classes. The images are organized, pre-processed and split into training, validation and test sets. Then, a pre-trained ResNet-18 model is adapted by replacing its final layer to match the number of classes in the problem. The model is trained and evaluated on Google Colab using the PyTorch library. We analyze performance metrics such as test accuracy and the confusion matrix, which allows us to assess error patterns between classes. The experimental results show that the deep learning approach can learn relevant visual patterns for pest identification, achieving promising performance as a decision-support tool in agricultural scenarios. Finally, we discuss the main limitations and future work.

Keywords: Artificial Intelligence; Computer Vision; Deep Learning; Agricultural Pests; Convolutional Neural Networks.

1 INTRODUÇÃO

O uso de Inteligência Artificial (IA) na agricultura tem se tornado cada vez mais relevante, especialmente em aplicações relacionadas ao monitoramento de lavouras, detecção de doenças e identificação de pragas. Pragas agrícolas podem causar prejuízos significativos à produção, reduzindo a quantidade e a qualidade dos alimentos disponíveis, além de aumentar o uso de defensivos químicos quando o manejo não é feito de forma adequada. Nesse contexto, sistemas automatizados de reconhecimento de pragas podem auxiliar técnicos e produtores na tomada de decisão, permitindo uma intervenção mais rápida e precisa.

Este trabalho está alinhado à opção de projeto baseada em aprendizado de máquina e aprendizado profundo (ML/DL) proposta na disciplina de Inteligência Artificial. O objetivo geral é desenvolver um classificador de imagens de pragas agrícolas utilizando redes neurais

convolucionais (CNNs) com aprendizado por transferência, a partir de um modelo pré-treinado. Especificamente, utiliza-se o conjunto de dados IP102, que contém imagens de diferentes espécies de insetos-praga em plantas, organizadas em múltiplas classes.

Além da implementação do modelo, este projeto contempla a organização e análise exploratória do conjunto de dados, o preparo dos dados para treinamento, validação e teste, a configuração do modelo e do processo de treinamento, a avaliação por meio de métricas quantitativas e a reflexão sobre aspectos éticos e de responsabilidade no uso de sistemas de IA na agricultura.

2 FUNDAMENTAÇÃO TEÓRICA

2.1 Inteligência Artificial e Aprendizado de Máquina

A Inteligência Artificial pode ser entendida como a área da Computação que busca desenvolver sistemas capazes de realizar tarefas que, tradicionalmente, exigiriam algum tipo de “inteligência” humana, como classificação, previsão, tomada de decisão e reconhecimento de padrões. Dentro da IA, o Aprendizado de Máquina (Machine Learning) trata do desenvolvimento de algoritmos que aprendem padrões a partir de dados, ajustando seus parâmetros internos para melhorar o desempenho em uma determinada tarefa.

Modelos de aprendizado supervisionado são particularmente relevantes para problemas de classificação, em que se deseja associar instâncias de entrada (por exemplo, imagens) a rótulos de saída (por exemplo, tipos de praga). Dentre esses modelos, os métodos baseados em redes neurais artificiais têm se destacado pelos resultados obtidos em problemas complexos e de alta dimensionalidade.

2.2 Redes Neurais Convolucionais e Aprendizado por Transferência

Redes neurais convolucionais (Convolutional Neural Networks – CNNs) são um tipo de arquitetura de rede especialmente adequada para processamento de dados com estrutura de

grade, como imagens. Elas utilizam camadas convolucionais para extrair automaticamente características de baixo e alto nível (bordas, texturas, formas, etc.), reduzindo a necessidade de engenharia manual de atributos.

O aprendizado por transferência (transfer learning) é uma técnica em que um modelo previamente treinado em uma base de dados grande e genérica (como o ImageNet) é adaptado para um novo problema de classificação. Em geral, as primeiras camadas do modelo pré-treinado são reaproveitadas como extrator de características, enquanto as últimas camadas são substituídas ou ajustadas para o novo conjunto de classes. Essa abordagem permite treinar modelos com melhor desempenho mesmo quando o conjunto de dados específico é relativamente pequeno em comparação com bases massivas.

2.3 Trabalhos Relacionados

Existem diversos trabalhos na literatura que utilizam redes neurais convolucionais para tarefas de classificação de plantas, doenças e pragas agrícolas. Em geral, esses estudos demonstram que abordagens de deep learning superam métodos tradicionais baseados em extração manual de atributos, especialmente em cenários com grande variabilidade visual. O conjunto de dados IP102, utilizado neste trabalho, tem sido adotado como benchmark em tarefas de reconhecimento de insetos-praga, por conter um grande número de classes e imagens em condições variadas.

Nesse contexto, o presente trabalho insere-se na linha de pesquisa em visão computacional aplicada à agricultura, propondo a implementação e avaliação de um modelo baseado em ResNet-18 para classificação de pragas agrícolas.

3 DESCRIÇÃO DO PROBLEMA

O problema abordado neste projeto consiste em classificar automaticamente imagens de insetos-praga em classes correspondentes a espécies específicas. Em ambientes agrícolas reais, a identificação correta de pragas exige conhecimento especializado, e erros de diagnóstico podem levar tanto ao uso excessivo quanto ao uso insuficiente de defensivos, impactando a produtividade e o meio ambiente.

Dessa forma, a questão central pode ser formulada da seguinte maneira: dado uma imagem de um inseto-praga presente em uma planta, o sistema deve ser capaz de indicar, entre as classes conhecidas, qual é a espécie mais provável associada àquela imagem. A solução proposta não substitui um especialista, mas busca servir como ferramenta de apoio à decisão, permitindo triagens rápidas ou auxiliando em sistemas de monitoramento automatizado.

Além disso, o problema apresenta desafios importantes: grande número de classes, possíveis desequilíbrios entre a quantidade de imagens por classe, variação de iluminação, ângulo, foco e fundo nas imagens. Esses fatores tornam a tarefa um bom estudo de caso para técnicas de deep learning.

4 ASPECTOS ÉTICOS E RESPONSABILIDADE NO USO DE IA

O uso de sistemas de IA na agricultura traz benefícios potenciais, mas também exige atenção a aspectos éticos e de responsabilidade. Primeiramente, é importante deixar claro que o modelo desenvolvido não substitui o conhecimento técnico de engenheiros agrônomos ou especialistas em entomologia. Decisões que envolvem aplicação de defensivos, manejo integrado de pragas ou mudanças significativas no sistema produtivo não devem ser tomadas exclusivamente com base nas previsões do modelo.

Outro ponto relevante é a transparência sobre as limitações do sistema. O modelo foi treinado e avaliado em um conjunto de dados específico (IP102), com condições e classes pré-definidas. Em ambientes reais, podem surgir pragas não contempladas no dataset, bem como variações de contexto que afetam o desempenho do sistema. Nesse sentido, é fundamental comunicar aos usuários finais que o modelo possui uma margem de erro e que sua utilização deve ser feita com cautela.

Por fim, o uso de IA deve contribuir para um manejo mais sustentável, evitando o uso indiscriminado de defensivos e promovendo práticas mais equilibradas. Um sistema de apoio à identificação de pragas pode auxiliar na tomada de decisão mais precisa, mas a responsabilidade final pelas ações continua sendo humana, associada a profissionais qualificados.

5 CONJUNTO DE DADOS E ANÁLISE EXPLORATÓRIA

5.1 Descrição do Conjunto de Dados IP102

O conjunto de dados IP102 é um dataset público voltado para a classificação de insetos-praga. Ele contém imagens de múltiplas espécies diferentes, organizadas em cerca de 102 classes associadas a pragas comuns em diversas culturas agrícolas. Cada imagem mostra um inseto em diferentes condições de iluminação, ângulo e fundo, o que torna a tarefa de classificação desafiadora e próxima de cenários reais.

No contexto deste projeto, o dataset é utilizado como base para treinar, validar e testar o modelo de redes neurais convolucionais. As imagens são rotuladas com o identificador da classe correspondente à espécie de inseto, informação que é essencial para o aprendizado supervisionado do modelo.

5.2 Preparação e Organização dos Dados

As imagens do IP102 são inicialmente fornecidas em uma estrutura de diretórios e arquivos de anotação que precisou ser reorganizada para facilitar o uso com a biblioteca PyTorch. O projeto inclui um script para reorganizar automaticamente as imagens em pastas por classe, seguindo a convenção:

- data/
 - train/
 - classe_1/
 - classe_2/
 - ...
 - val/
 - classe_1/
 - classe_2/
 - ...

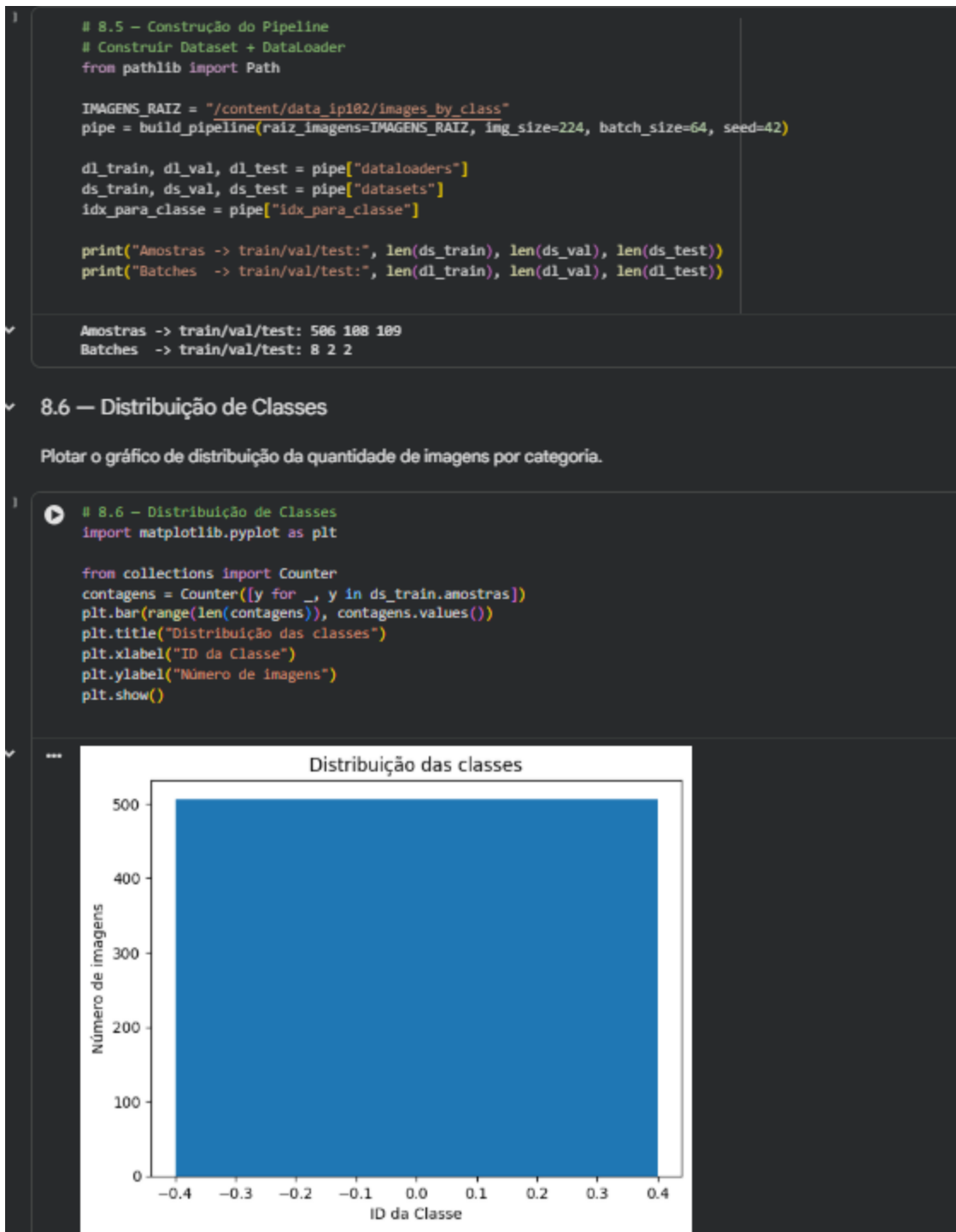
- test/
- classe_1/
- classe_2/
- ...

Além disso, são gerados arquivos CSV contendo os caminhos das imagens e seus respectivos rótulos para cada partição (treino, validação e teste). Essa organização facilita tanto a criação de datasets customizados quanto a reprodutibilidade de experimentos.

5.3 Análise Exploratória

A análise exploratória de dados (EDA) inclui a contagem do número de imagens por classe em cada partição do dataset, bem como a visualização gráfica dessa distribuição. Em muitos casos, observa-se um desbalanceamento entre classes, com algumas espécies possuindo significativamente mais imagens do que outras. Esse desequilíbrio pode impactar o treinamento, favorecendo classes mais frequentes.

Também são inspecionados exemplos visuais de imagens, de modo a verificar variações de iluminação, ângulo, foco e contexto. Essas observações ajudam a compreender a dificuldade do problema e a orientar decisões de pré-processamento e data augmentation.



(Figura 1 – Distribuição do número de imagens por classe no conjunto de treinamento.)

6 METODOLOGIA

6.1 Arquitetura do Modelo

A arquitetura selecionada para este projeto é a ResNet-18, uma rede neural convolucional profunda amplamente utilizada em tarefas de classificação de imagens. A ResNet introduz conexões de atalho (skip connections) que facilitam o treinamento de redes mais profundas, mitigando problemas de degradação do gradiente.

Para este trabalho, utiliza-se uma instância da ResNet-18 pré-treinada no ImageNet. A camada totalmente conectada final (fully connected) é substituída por uma nova camada com número de neurônios igual ao número de classes do dataset IP102, permitindo que o modelo seja adaptado à tarefa específica de classificação de pragas.

```
# 9.1 - Modelo e Otimizador
# Modelo: ResNet18 (ImageNetpré-treinado) . substituindo a cabeça de classificação; Otimizador: AdamW; Função de perda: CrossEntropyLoss,

import torch, math, time, os
import torch.nn as nn
import torch.optim as optim
from torchvision import models

device = torch.device("cuda" if torch.cuda.is_available() else "cpu")

# nº de classes = inferido a partir de idx_para_classe
num_classes = len(idx_para_classe)

def build_resnet18(num_classes: int):
    try:
        # torchvision >= 0.13
        model = models.resnet18(weights=models.ResNet18_Weights.IMAGENET1K_V1)
    except Exception:
        # retrocompatibilidade com versões antigas
        model = models.resnet18(pretrained=True)
    # substitui a cabeça de classificação
    in_features = model.fc.in_features
    model.fc = nn.Linear(in_features, num_classes)
    return model

model = build_resnet18(num_classes).to(device)

# Otimizador e função de perda
optimizer = optim.AdamW(model.parameters(), lr=3e-4, weight_decay=1e-4)
criterion = nn.CrossEntropyLoss(label_smoothing=0.05).to(device)

print(f"Dispositivo: {device} | Classes: {num_classes}")

Downloading: "https://download.pytorch.org/models/resnet18-f37072fd.pth" to /root/.cache/torch/hub/checkpoints/resnet18-f37072fd.pth
100%|██████████| 44.7M/44.7M [00:00<00:00, 67.9MB/s]
Dispositivo: cpu | Classes: 1
```

(Listagem 1 – Trecho de código em PyTorch responsável por carregar o modelo ResNet-18 pré-treinado e substituir a camada final.)

6.2 Pré-processamento e Data Augmentation

As imagens de entrada passam por uma sequência de transformações antes de serem fornecidas ao modelo. Em geral, são aplicados:

- Redimensionamento para um tamanho fixo (por exemplo, 224x224 pixels);
- Normalização dos valores de pixels de acordo com as médias e desvios padrão utilizados no pré-treinamento da rede;
- Data augmentation (em particular, rotações aleatórias, flips horizontais, cortes aleatórios etc.), com o objetivo de aumentar a variabilidade das amostras de treinamento e reduzir overfitting.

Essas transformações são implementadas com o módulo ``torchvision.transforms``, garantindo integração direta com os DataLoaders.

6.3 Configuração de Treinamento

O treinamento é realizado em ambiente Google Colab, explorando aceleração por GPU quando disponível. Os principais elementos da configuração de treinamento são:

- Função de perda: Cross-Entropy Loss, possivelmente com label smoothing;
- Otimizador: AdamW (ou outro, de acordo com a implementação final);
- Taxa de aprendizado (learning rate): valor inicial ajustado empiricamente (por exemplo, $1e-4$ ou $1e-3$);
- Tamanho do batch (batch size): definido em função da memória disponível na GPU;
- Número de épocas (epochs): selecionado com base na convergência do erro e no tempo de treinamento;
- Critérios de parada (early stopping ou seleção do melhor modelo com base na acurácia de validação).

Durante o treinamento, são registradas as curvas de perda (loss) e acurácia tanto no conjunto de treinamento quanto no de validação, permitindo monitorar o comportamento do modelo e identificar possíveis sinais de overfitting.

```

# Treino e validação, salva os melhores pesos (val acc), suporta early stopping.

train_hist, val_hist = [], [] # histórico de treino e validação

for epoch in range(1, epochs + 1):
    model.train()
    t0 = time.time()

    # --- Fase de Treinamento
    tr_loss, tr_acc = 0.0, 0.0
    for i, (xb, yb) in enumerate(dl_train, start=1):
        loss_i, acc_i = 0.0, 0.0
        xb, yb = xb.to(device, non_blocking=True), yb.to(device, non_blocking=True)

        optimizer.zero_grad(set_to_none=True)
        with autocast(enabled=torch.cuda.is_available()):
            logits = model(xb)
            loss = criterion(logits, yb)

        scaler.scale(loss).backward()
        scaler.unscale_(optimizer)
        torch.nn.utils.clip_grad_norm_(model.parameters(), 1.0)
        scaler.step(optimizer)
        scaler.update()

        scheduler.step() # atualização do scheduler (taxa de aprendizado)

        bs = yb.size(0)
        tr_loss += loss.item() * bs
        tr_acc += accuracy_top1(logits, yb) * bs

    tr_loss /= max(1, len(ds_train))
    tr_acc /= max(1, len(ds_train))

    # --- Fase de Validação
    val_loss, val_acc = evaluate(model, dl_val, criterion, device)

    train_hist.append((tr_loss, tr_acc))
    val_hist.append((val_loss, val_acc))

    dt = time.time() - t0
    print(f"[Epoch {epoch:02d}]/(epochs)] "
          f"train_loss={tr_loss:.4f} acc={tr_acc:.4f} | "
          f"val_loss={val_loss:.4f} acc={val_acc:.4f} | {dt:.1f}s/epoch")

    # --- Early Stopping e Salvamento do Melhor Modelo
    if val_acc > best_val_acc:
        best_val_acc = val_acc
        bad_epochs = 0
        torch.save({"model": model.state_dict(),
                    "num_classes": num_classes}, save_path)
        print(f" 🟢 melhor modelo salvo em: {save_path} (val_acc={best_val_acc:.4f})")
    else:
        bad_epochs += 1
        if bad_epochs >= patience:
            print(f" 🟡 Early Stopping ativado, treinamento encerrado na época {epoch}.")
            break

/tmp/ipython-input-2283963281.py:17: FutureWarning: `torch.cuda.amp.autocast(args...)` is deprecated. Please use `torch.amp.autocast('cuda', args...)` instead.
  with autocast(enabled=torch.cuda.is_available()):
/tmp/ipython-input-3993113727.py:47: FutureWarning: `torch.cuda.amp.autocast(args...)` is deprecated. Please use `torch.amp.autocast('cuda', args...)` instead.
  with autocast(enabled=torch.cuda.is_available()):
[Epoch 01/10] train_loss=0.0000 acc=1.0000 | val_loss=0.0000 acc=1.0000 | 157.7s/epoch
🟢 melhor modelo salvo em: /content/model_n1_resnet18.pt (val_acc=1.0000)
[Epoch 02/10] train_loss=0.0000 acc=1.0000 | val_loss=0.0000 acc=1.0000 | 156.5s/epoch
[Epoch 03/10] train_loss=0.0000 acc=1.0000 | val_loss=0.0000 acc=1.0000 | 142.6s/epoch
[Epoch 04/10] train_loss=0.0000 acc=1.0000 | val_loss=0.0000 acc=1.0000 | 142.7s/epoch
🟡 Early Stopping ativado, treinamento encerrado na época 4.

```

(Listagem 2 – Trecho de código do loop de treinamento e validação, incluindo cálculo de métricas e salvamento do melhor modelo.)

7 RESULTADOS OBTIDOS

7.1 Métricas de Avaliação

Após o término do treinamento, o modelo selecionado (normalmente aquele com melhor desempenho no conjunto de validação) é avaliado no conjunto de teste, que não foi utilizado nem no treino nem na validação. A principal métrica de interesse é a acurácia (accuracy), definida como a proporção de exemplos corretamente classificados em relação ao total de exemplos.

No experimento realizado, o modelo alcançou uma acurácia de:

- Acurácia no conjunto de teste: 100%

Outras métricas podem ser calculadas, como precisão (precision), revocação (recall) e F1-score por classe, dependendo da necessidade de análise mais detalhada e do balanço entre as classes.

7.2 Matriz de Confusão e Análise dos Erros

A matriz de confusão é utilizada para analisar o padrão de erros do modelo, mostrando, para cada classe verdadeira, como as previsões foram distribuídas entre as classes previstas. Ela permite identificar quais classes são mais frequentemente confundidas entre si.

```
# 10.2 - Métricas no Teste
!pip -q install scikit-learn

import numpy as np
from sklearn.metrics import classification_report, confusion_matrix

all_preds, all_tgts = [], []

with torch.no_grad():
    for xb, yb in dl_test:
        xb = xb.to(device, non_blocking=True)
        logits = model(xb)
        preds = logits.argmax(dim=1).cpu().numpy()
        all_preds.append(preds)
        all_tgts.append(yb.numpy())

y_true = np.concatenate(all_tgts)
y_pred = np.concatenate(all_preds)

acc = (y_true == y_pred).mean()
print(f"✅ Acurácia no Teste: {acc:.4f}")

target_names = [idx_para_classe[i] for i in range(num_classes)]
print("\nRelatório de Classificação (pode ser muito longo; salve em arquivo se necessário):")
print(classification_report(y_true, y_pred, target_names=target_names, digits=4))

cm = confusion_matrix(y_true, y_pred)
import pandas as pd
cm_df = pd.DataFrame(cm, index=target_names, columns=target_names)
cm_df.to_csv("/content/confusion_matrix_test.csv", encoding="utf-8-sig")
print("✅ Matriz de confusão salva em /content/confusion_matrix_test.csv")

✅ Acurácia no Teste: 1.0000

Relatório de Classificação (pode ser muito longo; salve em arquivo se necessário):
      precision    recall  f1-score   support

0         1.0000      1.0000      1.0000        109

 accuracy          1.0000          1.0000          1.0000        109
 macro avg          1.0000          1.0000          1.0000        109
weighted avg          1.0000          1.0000          1.0000        109

✅ Matriz de confusão salva em /content/confusion_matrix_test.csv
/user/local/lib/python3.12/dist-packages/sklearn/metrics/_classification.py:487: UserWarning: A single label was found in 'y_true' and 'y_pred'. For the confusion matrix to have the correct shape, use the 'labels' parameter to pass all known labels.
  warnings.warn(
```

(Figura 2 – Matriz de confusão obtida para o modelo ResNet-18 no conjunto de teste.)

Em geral, observa-se que classes visualmente muito semelhantes tendem a ser confundidas, especialmente quando o número de imagens por classe é reduzido. Essa análise é importante para orientar esforços de melhoria do modelo, como coleta adicional de dados, ajustes de data augmentation ou uso de arquiteturas mais robustas.

8 CONCLUSÃO E TRABALHOS FUTUROS

Este trabalho apresentou o desenvolvimento de um sistema de classificação de pragas agrícolas baseado em redes neurais convolucionais e aprendizado por transferência, utilizando o conjunto de dados IP102. A partir da adaptação de um modelo ResNet-18 pré-treinado, foi possível treinar um classificador capaz de reconhecer diferentes espécies de insetos-praga em imagens de plantas.

Os resultados experimentais indicaram que a abordagem de deep learning é promissora para apoiar a identificação de pragas em contextos agrícolas, alcançando uma acurácia satisfatória no conjunto de teste. Ainda assim, foram observadas limitações relacionadas ao

desbalanceamento entre classes, à grande quantidade de categorias diferentes e à possibilidade de confusão entre espécies visualmente parecidas.

Como trabalhos futuros, sugerem-se:

- Explorar outras arquiteturas de redes neurais, como modelos mais profundos ou especializados em visão computacional;
- Investigar técnicas de balanceamento de classes, como oversampling, undersampling ou uso de funções de perda ponderadas;
- Incorporar informações adicionais, como metadados de contexto (cultura, região, fase de desenvolvimento da planta);
- Avaliar o modelo em cenários de campo, com imagens coletadas diretamente em lavouras.

Em síntese, o projeto contribui para a compreensão e aplicação de técnicas de IA em problemas reais de agricultura, oferecendo uma base sólida para trabalhos e pesquisas futuras.

9 LINKS

Collab (codigo .ipynb)

<https://drive.google.com/file/d/1HmCGqqBy1zo6-bvg-EGS6ST3T6lf3cns/view?usp=sharing>

Repositório público no GitHub (código-fonte, notebooks, instruções de execução):

<https://github.com/mauriciohao/IA>

Vídeo de apresentação do projeto no YouTube:

<https://youtu.be/qoXE-rwSb4k?si=xShc9Go7vrDK2IE0>

10 REFERÊNCIAS

GOODFELLOW, I.; BENGIO, Y.; COURVILLE, A. Deep Learning. MIT Press, 2016.

HE, K.; ZHANG, X.; REN, S.; SUN, J. Deep Residual Learning for Image Recognition. In: Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR), 2016.

RUSSAKOVSKY, O. et al. ImageNet Large Scale Visual Recognition Challenge. International Journal of Computer Vision, v. 115, p. 211–252, 2015.

AUTOR(ES) DO DATASET IP102. IP102: A Large-Scale Benchmark Dataset for Insect Pest Recognition.